

Person 3 — AI Core

**Role: OpenRouter LLM · Diagnosis JSON · ElevenLabs TTS ·
Prompt Engineering**

You are the brain of the app. Nothing works without your API calls.

Your first job at Hour 0 is collecting ALL API keys.

Your Hours

Hours	Task
0–1	Collect ALL API keys, create <code>.env</code> , share with team
1–3	OpenRouter account → first test call in browser console
7–11	Full LLM pipeline: prompt → fetch → JSON parse → error handling
11–13	ElevenLabs TTS — read diagnosis aloud
12–14	Severity → colour mapping logic
24–26	Edge case testing: timeout, bad JSON, emergency cases

Step 0: Collect ALL API Keys (Hour 0 — Do This First)

Everyone needs these. Do it before writing a single line of code.

OpenRouter (Free — access to Claude + Qwen)

1. Go to openrouter.ai
2. Click "Sign In" → use Google account
3. Go to **Keys** tab → "Create Key"
4. Copy the key (starts with `sk-or-`)
5. Go to **Credits** → add \$5 minimum (or check if LotusHacks gives free credits)
 - At hackathons, OpenRouter often provides free credits — ask organisers
 - Claude Sonnet costs ~\$0.003/request, so \$5 = ~1,600 test calls

ElevenLabs (Free tier)

1. Go to elevenlabs.io

Step 1: System Prompt (Hours 7–8)

This is the most important thing you will write. Iterate on it.

```
// api/openrouter.js

const SYSTEM_PROMPT_EN = `You are a medical triage AI assistant for a Vietnamese healthcare app.
Analyze the patient's selected body region and symptoms, then respond ONLY with valid JSON.
No markdown, no backticks, no preamble. Raw JSON only.

Response format:
{
  "conditions": [
    {
      "name": "Condition name",
      "likelihood": "High|Medium|Low",
      "explanation": "One clear sentence explaining why this condition fits."
    }
  ],
  "severity": "Low|Medium|High|Emergency",
  "action": "One clear action sentence.",
  "home_tips": [
    "Specific tip 1",
    "Specific tip 2"
  ],
  "warning_signs": [
    "Symptom that would require immediate emergency care"
  ],
  "disclaimer": "This is AI-generated information only and not medical advice. Always consult a qualified doctor. For emergencies in Vietnam, call 115."
}
```

Rules:

- Provide exactly 2 to 3 conditions, ranked by likelihood (most likely first)
- severity "Emergency" ONLY when there is real risk to life: signs of appendicitis, heart attack, stroke, severe bleeding, difficulty breathing
- Keep each explanation under 25 words
- home_tips: 2 to 3 practical things the patient can do right now
- warning_signs: 1 to 3 specific red flags to watch for
- Always use the disclaimer field exactly as shown, do not modify it
- Respond in English;

```
const SYSTEM_PROMPT_VI = `Bạn là trợ lý AI phân loại y tế cho ứng dụng chăm sóc sức khỏe Việt Nam.
Phân tích vùng cơ thể và triệu chứng của bệnh nhân, sau đó chỉ trả lời bằng JSON hợp lệ.
Không có markdown, không có backtick, không có lời mở đầu. Chỉ JSON thuần túy.
```

```
Định dạng phản hồi:
{
  "conditions": [
    {
      "name": "Tên bệnh",
      "likelihood": "High|Medium|Low",
      "explanation": "Một câu giải thích rõ ràng."
    }
  ],
  "severity": "Low|Medium|High|Emergency",
  "action": "Một câu hành động rõ ràng.",
  "home_tips": ["Mẹo cụ thể 1", "Mẹo cụ thể 2"],
  "warning_signs": ["Triệu chứng cần cấp cứu ngay"],
  "disclaimer": "Đây chỉ là thông tin AI và không phải lời khuyên y tế. Luôn tham khảo bác sĩ. Cấp cứu tại Việt Nam: gọi 115."
}
```

Quy tắc:

Step 2: OpenRouter API Call (Hours 8–11)

```
// api/openrouter.js (continued)

const OPENROUTER_KEY = window.ENVY.OPENROUTER_KEY || '';

const MODELS = {
  en: 'anthropic/claude-3.5-sonnet', // Best for English medical
  vi: 'qwen/qwen-2.5-72b-instruct', // Best for Vietnamese
};

// Fallback model if primary fails
const FALLBACK_MODEL = 'openai/gpt-4o-mini';

export async function callDiagnosis(userMessage, language = 'en') {
  const model = MODELS[language] || MODELS.en;
  const systemPrompt = language === 'vi' ? SYSTEM_PROMPT_VI : SYSTEM_PROMPT_EN;

  const body = {
    model: model,
    max_tokens: 1000,
    messages: [
      { role: 'system', content: systemPrompt },
      { role: 'user', content: userMessage }
    ],
    response_format: { type: 'json_object' }, // Forces JSON mode
  };

  try {
    const response = await fetch('https://openrouter.ai/api/v1/chat/completions', {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${OPENROUTER_KEY}`,
        'HTTP-Referer': window.location.origin,
        'X-Title': 'BodyCheck - LotusHacks 2026',
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(body),
    });

    if (!response.ok) {
      const err = await response.json();
      throw new Error('OpenRouter error: ' + JSON.stringify(err));
    }

    const data = await response.json();
    const rawText = data.choices?.[0]?.message?.content;

    if (!rawText) throw new Error('Empty response from model');

    // Clean up any accidental markdown wrapping
    const cleaned = rawText
      .replace(/```json/g, '')
      .replace(/```/g, '')
      .replace(/"/g, '')
      .trim();

    const parsed = JSON.parse(cleaned);

    // Validate required fields
    if (!parsed.conditions || !parsed.severity || !parsed.action) {
      throw new Error('Incomplete diagnosis JSON from model');
    }

    return { success: true, data: parsed };
  } catch (err) {
    console.error('Diagnosis error:', err);

    // Try fallback model once
    if (body.model !== FALLBACK_MODEL) {
      console.warn('Retrying with fallback model:', FALLBACK_MODEL);
      body.model = FALLBACK_MODEL;
      try {
        const r2 = await fetch('https://openrouter.ai/api/v1/chat/completions', {
          method: 'POST',
          headers: {
            'Authorization': `Bearer ${OPENROUTER_KEY}`,
            'Content-Type': 'application/json',
          },
          body: JSON.stringify(body),
        });
        const d2 = await r2.json();
        const t2 = d2.choices?.[0]?.message?.content?.trim();
        if (t2) return { success: true, data: JSON.parse(t2) };
      } catch (e2) {
        console.error('Fallback also failed:', e2);
      }
    }

    return {
      success: false,
      error: err.message,
      data: fallbackDiagnosis(language)
    };
  }
}

// Graceful fallback if all API calls fail
function fallbackDiagnosis(lang) {
  if (lang === 'vi') {
    return {
      conditions: [{ name: 'thông tin chưa rõ ràng', likelihood: 'Low',
        explanation: 'Vui lòng thử lại hoặc tham khảo bác sĩ trực tiếp.' }],
      severity: 'Medium',
      action: 'Vui lòng tham khảo bác sĩ hoặc cơ sở y tế gần nhất.',
      home_tips: ['Nghỉ ngơi và theo dõi triệu chứng', 'Ghi lại thời gian và mức độ đau'],
      warning_signs: ['Đau tăng đột ngột', 'Mất ý thức', 'Mệt y quá mức'],
      disclaimer: 'Đây chỉ là thông tin AI. Luôn tham khảo bác sĩ. Cấp cứu: gọi 115.'
    };
  }
  return {
    conditions: [{ name: 'Unable to analyse', likelihood: 'Low',
      explanation: 'Please try again or consult a medical professional directly.' }],
    severity: 'Medium',
    action: 'Please consult a doctor or visit your nearest medical facility.',
    home_tips: ['Rest and monitor your symptoms', 'Note the time and intensity of pain'],
    warning_signs: ['Sudden worsening of pain', 'Difficulty breathing', 'Loss of consciousness'],
    disclaimer: 'This is AI information only. Always consult a qualified doctor. Emergencies: call 115.'
  };
}
```

Step 3: Wire runDiagnosis (Hours 10–11)

This is the global function Person 2 calls when the form is submitted. It orchestrates all API calls in parallel.

```
// In index.html <script type="module">
import { callDiagnosis } from './api/openrouter.js';
import { speakText } from './api/elevenlabs.js';
import { renderDiagnosisCard } from './components/diagnosis-card.js';

window.runDiagnosis = async function(userMessage, language) {
  state.setLoading(true);
  document.getElementById('diagnosis-card').style.display = 'none';

  // Fire all AI calls in parallel
  const [diagnosisResult] = await Promise.all([
    callDiagnosis(userMessage, language),
    // fal.ai and Exa.ai calls (Person 4) are wired here too
    window.generateBodyImage?.(state.selectedLabel),
    window.fetchMedicalSources?.(),
  ]);

  state.setLoading(false);

  const diagData = diagnosisResult.data;
  renderDiagnosisCard(diagData);

  // TTS – read the diagnosis
  const summary = `${diagData.conditions[0]?.name}. ${diagData.action}`;
  speakText(summary, language);
}
```

Step 4: ElevenLabs TTS (Hours 11–13)

```
// api/elevenlabs.js

const ELEVENLABS_KEY = window.ENV?.ELEVENLABS_KEY || '';

// Voice IDs - test these and pick what sounds best
const VOICES = {
  en: 'EXAVITQu4vr4xnSDxMal', // "Sarah" - clear, neutral
  vi: 'EXAVITQu4vr4xnSDxMal', // Multilingual v2 model supports Vietnamese
};

export async function speakText(text, language = 'en') {
  if (!text || !ELEVENLABS_KEY) return;

  const ttsBtn = document.getElementById('tts-btn');
  if (ttsBtn) {
    ttsBtn.textContent = '🔊 Playing...';
    ttsBtn.disabled = true;
  }

  try {
    const voiceId = VOICES[language] || VOICES.en;

    const response = await fetch(
      `https://api.elevenlabs.io/v1/text-to-speech/${voiceId}`,
      {
        method: 'POST',
        headers: {
          'xi-api-key': ELEVENLABS_KEY,
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({
          text: text,
          model_id: 'eleven_multilingual_v2', // Supports Vietnamese
          voice_settings: {
            stability: 0.5,
            similarity_boost: 0.8,
            style: 0.0,
            use_speaker_boost: true,
          }
        })
      }
    );

    if (!response.ok) {
      const err = await response.text();
      throw new Error(`TTS failed: ${err}`);
    }

    const audioBlob = await response.blob();
    const audioUrl = URL.createObjectURL(audioBlob);
    const audio = new Audio(audioUrl);

    audio.onended = () => {
      URL.revokeObjectURL(audioUrl);
      if (ttsBtn) {
        ttsBtn.textContent = '🔊 Read aloud';
        ttsBtn.disabled = false;
      }
    };

    await audio.play();
  } catch (err) {
    console.error('TTS error:', err);
    if (ttsBtn) {
      ttsBtn.textContent = '🔊 Read aloud';
      ttsBtn.disabled = false;
    }
  }
}

// Allow manual trigger from TTS button
export function initTTSButton(text, language) {
  const btn = document.getElementById('tts-btn');
  if (btn) {
    btn.onclick = () => speakText(text, language);
  }
}
```

Prompt Testing (Before Integrating)

Test your prompt in the OpenRouter Playground first:

- 1. Go to **openrouter.ai/playground**
- 2. Select model: `anthropic/claude-3.5-sonnet`
- 3. Add your system prompt
- 4. Test with: `Body region: Lower right abdomen. Pain type: sharp. Duration: 6 hours. Severity: 8. Other: fever, nausea`
- 5. Verify JSON structure matches the expected format

Good test cases to verify:

Input	Expected severity
Lower right abdomen + sharp + fever + 8/10	High or Emergency (appendicitis)
Head + dull + 3 days + 4/10	Low or Medium (tension headache)

Your Deliverables Checklist

- [] `.env.example` committed to repo
- [] `config.js` created locally (in `.gitignore`)
- [] API keys distributed to whole team via secure chat
- [] `api/openrouter.js` — system prompts (EN + VI), fetch, JSON parse, fallback
- [] `api/elevenlabs.js` — TTS playback, multilingual support
- [] `window.runDiagnosis()` exported to global scope
- [] Prompt tested in OpenRouter playground with 4+ test cases
- [] Emergency case correctly returns severity "Emergency"

When You're Stuck

- JSON parse fails → add `console.log('raw:', rawText)` before parse to see what the model returned
- 401 error → API key wrong or not loaded (check `window.ENV`)
- Model returns English when Vietnamese requested → add stronger instruction to system prompt: "You MUST respond entirely in Vietnamese"
- TTS plays no audio → browser autoplay policy — add a "Play" button user must click first
- Rate limit hit → switch to `openai/gpt-4o-mini` as fallback (much cheaper)